Module 4 — Pandas Basics

Two sections:

1. **Fundamentals** — creating DataFrames, indexing, filtering, basic operations
2. **Key operations** — groupby, merge, apply, reshape

Each section: read the concept, try the exercise, check the solution.

In [1]:
```
import pandas as pd
import numpy as np
```

Part 1 — Fundamentals

Creating a DataFrame

```
# From a dict — keys are column names
df = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]})

# From a list of lists
df = pd.DataFrame([[1, 4], [2, 5], [3, 6]], columns=['a', 'b'])
```

Inspecting

```
df.shape        # (nrows, ncols)
df.dtypes       # column types
df.head(3)      # first 3 rows
df.describe()   # summary stats
df.columns      # Index of column names
df.index        # row index
```

Selecting columns

```
df['a']           # Series (one column)
df[['a', 'b']]    # DataFrame (multiple columns)
```

Selecting rows

```
df.iloc[0]        # row by position (int)
df.iloc[0:3]      # rows 0,1,2 by position
df.loc[0]         # row by label (index value)
df.loc[0:2]       # rows with index 0,1,2 (INCLUSIVE)
```

Key difference: `iloc` uses position (exclusive end like Python), `loc` uses labels (inclusive end).

Exercise 1 — Creating and inspecting

Create a DataFrame with columns `name`, `age`, `score` from the data below. Then:

- Print its shape
- Select just the `score` column
- Select the first 2 rows using `iloc`

```
data = [
    ['Alice', 25, 88],
    ['Bob',   30, 72],
    ['Carol', 22, 95],
    ['Dave',  28, 60],
```

```
]
```

In [65]:
```
data = [
    ['Alice', 25, 88],
    ['Bob',   30, 72],
    ['Carol', 22, 95],
    ['Dave',  28, 60],
]

# implement this
df = pd.DataFrame(data, columns=["name","age","score"])

# print shape, score column, first 2 rows
print(df.shape)
print(df["score"])
print(df.iloc[:2])
```

```
(4, 3)
0    88
1    72
2    95
3    60
Name: score, dtype: int64
    name  age  score
0  Alice   25     88
1    Bob   30     72
```

▸ Solution

```
df = pd.DataFrame(data, columns=['name', 'age', 'score'])
print(df.shape)        # (4, 3)
print(df['score'])     # Series
print(df.iloc[:2])     # first 2 rows
```

Filtering (boolean indexing)

```
df[df['age'] > 25]                          # rows where age > 25
df[(df['age'] > 25) & (df['score'] > 70)] # AND — must use & not
'and', wrap in parens
df[df['name'].isin(['Alice', 'Bob'])]      # membership
df[df['score'].isna()]                      # null values
```

Gotcha: use `&` / `|` not `and` / `or`. Always wrap conditions in parentheses.

Exercise 2 — Filtering

Using the same `df`:

- Filter to rows where `age >= 25`
- Filter to rows where `age >= 25 AND score >= 70`
- Return just the `name` column of the filtered result

In [26]:
```
# implement this
filtered = df[(df["age"]>=25) & (df["score"]>70) ]
names = filtered["name"]
print(names)
```

```
0    Alice
1      Bob
Name: name, dtype: str
```

▸ Solution

```
filtered = df[(df['age'] >= 25) & (df['score'] >= 70)]
names = filtered['name']
print(names)  # Alice, Bob
```

Adding and modifying columns

```
df['grade'] = df['score'] / 10        # new column from arithmetic
df['pass'] = df['score'] >= 75        # boolean column
df['score'] = df['score'] + 5         # modify in place
df.drop(columns=['grade'], inplace=True)  # remove column
```

Sorting

```
df.sort_values('score')                      # ascending
df.sort_values('score', ascending=False)     # descending
df.sort_values(['age', 'score'])             # multi-key
```

Aggregations on a Series

```
df['score'].mean()
df['score'].sum()
df['score'].max()
df['score'].idxmax()   # index of the max value
df['score'].value_counts()  # frequency of each value
```

Exercise 3 — Column ops and aggregation

Using `df`:

- Add a column `pass` that is `True` if `score >= 75`
- Find the mean score of people who passed
- Find the name of the person with the highest score

In [32]:
```
# implement this
df["pass"] = df["score"] >= 75
print(df[df["pass"]]["score"].mean())
print(df.iloc[df["score"].idxmax()]["name"])
```

```
91.5
Carol
```

▸ Solution

```python
df['pass'] = df['score'] >= 75
mean_pass = df[df['pass']]['score'].mean()   # 91.5
best = df.loc[df['score'].idxmax(), 'name']  # 'Carol'
```

Part 2 — Key Operations

groupby

Split → apply → combine.

```python
df.groupby('dept')['salary'].mean()     # mean salary per dept
df.groupby('dept')['salary'].agg(['mean', 'max', 'count'])
df.groupby(['dept', 'level'])['salary'].sum()  # multi-key groupby
```

Result is a Series or DataFrame indexed by the group key(s).

Exercise 4 — groupby

```python
employees = pd.DataFrame({
    'name':   ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'dept':   ['eng',   'eng', 'hr',    'hr',   'eng'],
    'salary': [120,     95,    80,      85,      110],
})
```

- Compute mean salary per department
- Find the department with the highest mean salary
- Count how many employees are in each department

In [56]:
```python
employees = pd.DataFrame({
    'name':   ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'dept':   ['eng',   'eng', 'hr',    'hr',   'eng'],
    'salary': [120,     95,    80,      85,      110],
})

# implement this
```

In [57]:
```python
employees.groupby('dept')['salary'].agg(['mean', 'max', 'count'])
```

Out[57]:

dept	mean	max	count
eng	108.333333	120	3
hr	82.500000	85	2

In [58]:
```
employees["dept"].value_counts()
```

Out[58]:
```
dept
eng    3
hr     2
Name: count, dtype: int64
```

▶ Solution

```
mean_sal = employees.groupby('dept')['salary'].mean()
# dept
# eng    108.33
# hr      82.50

best_dept = mean_sal.idxmax()   # 'eng'

counts = employees.groupby('dept')['name'].count()
# or: employees['dept'].value_counts()
```

---

merge (join)

```
pd.merge(df1, df2, on='key')                    # inner join on column 'key'
pd.merge(df1, df2, on='key', how='left')        # left join
pd.merge(df1, df2, left_on='id', right_on='emp_id')  # different column names
```

- **inner**: only rows with matching keys in both (default)
- **left**: all rows from left, NaN where no match in right
- **outer**: all rows from both

Exercise 5 — merge

```
orders = pd.DataFrame({
    'order_id':   [1, 2, 3, 4],
    'customer_id':[10, 11, 10, 12],
    'amount':     [50, 30, 20, 80],
})
customers = pd.DataFrame({
    'customer_id': [10, 11, 13],
    'name':        ['Alice', 'Bob', 'Carol'],
})
```

- Inner join to get each order with the customer name
- How many orders are lost in the inner join vs a left join?

In [60]:
```
orders = pd.DataFrame({
    'order_id':    [1, 2, 3, 4],
    'customer_id': [10, 11, 10, 12],
    'amount':      [50, 30, 20, 80],
```

```
})
customers = pd.DataFrame({
    'customer_id': [10, 11, 13],
    'name':        ['Alice', 'Bob', 'Carol'],
})

pd.merge(orders, customers, left_on="order_id", right_on="customer_id")
```

Out[60]:

order_id	customer_id_x	amount	customer_id_y	name

In [62]:
```
pd.merge(orders, customers, on="customer_id", how="left")
```

Out[62]:

	order_id	customer_id	amount	name
0	1	10	50	Alice
1	2	11	30	Bob
2	3	10	20	Alice
3	4	12	80	NaN

▸ Solution

```
inner = pd.merge(orders, customers, on='customer_id')         # 3 rows (order 4 lost)
left  = pd.merge(orders, customers, on='customer_id', how='left')  # 4 rows (order 4 kept, name=NaN)
print(len(left) - len(inner))  # 1 order lost
```

---

apply

Apply an arbitrary function to each row or column.

```
df['score'].apply(lambda x: 'pass' if x >= 75 else 'fail')  # element-wise on Series
df.apply(lambda row: row['score'] * 2, axis=1)              # row-wise on DataFrame
```

For simple arithmetic, prefer vectorized operations (`df['score'] * 2`) — `apply` is slower but handles arbitrary logic.

Exercise 6 — apply

Using `df` from Exercise 1:

- Add a column `grade` that maps score to a letter: A ≥ 90, B ≥ 75, C ≥ 60, F otherwise
- Use `apply`

In [66]:
```
# implement this
```

```
def score_to_grade(score):
    grade = "F"
    if score >= 60:
        grade = "C"
    if score >= 75:
        grade = "B"
    if score >= 90:
        grade = "A"
    return grade

df["grade"]=df["score"].apply(score_to_grade)
```

In [67]:
```
df
```

Out[67]:

	name	age	score	grade
0	Alice	25	88	B
1	Bob	30	72	C
2	Carol	22	95	A
3	Dave	28	60	C

▸ Solution

```
def letter(score):
    if score >= 90: return 'A'
    if score >= 75: return 'B'
    if score >= 60: return 'C'
    return 'F'

df['grade'] = df['score'].apply(letter)
```

Reshaping — pivot and melt

pivot: long → wide

Each (student, subject) pair becomes a cell.

```
scores = pd.DataFrame({
    'student': ['Alice', 'Alice', 'Bob', 'Bob'],
    'subject': ['math',  'english', 'math', 'english'],
    'score':   [90, 85, 70, 80],
})

scores.pivot(index='student', columns='subject', values='score')
# subject  english  math
# student
# Alice         85    90
# Bob           80    70
```

melt: wide → long (inverse of pivot)

```
wide = pd.DataFrame({
    'student': ['Alice', 'Bob'],
    'math':    [90, 70],
    'english': [85, 80],
})

wide.melt(id_vars=['student'], var_name='subject',
value_name='score')
#   student  subject  score
# 0   Alice     math     90
# 1     Bob     math     70
# 2   Alice  english     85
# 3     Bob  english     80
```

`id_vars` = columns to keep as-is; everything else gets stacked into rows.

Missing values

```
df.isna().sum()              # count nulls per column
df.dropna()                  # drop rows with any null
df.fillna(0)                 # fill nulls with 0
df['col'].fillna(df['col'].mean())  # fill with column mean
```

In [2]:
```
import pandas as pd
```

In [6]:
```
scores = pd.DataFrame({
    'student': ['Alice', 'Alice', 'Bob', 'Bob'],
    'subject': ['math',  'english', 'math', 'english'],
    'score':   [90, 85, 70, 80],
})
```

In [7]:
```
scores
```

Out[7]:

	student	subject	score
0	Alice	math	90
1	Alice	english	85
2	Bob	math	70
3	Bob	english	80

In [14]:
```
scores_new=scores.pivot(index='student', columns='subject', values='score')
```

In [15]:
```
scores_new
```

Out[15]:

subject	english	math
student		
Alice	85	90
Bob	80	70

In [24]:
```
scores_new.iloc[0]
```

Out[24]:
```
subject
english    85
math       90
Name: Alice, dtype: int64
```

In [8]:
```
wide = pd.DataFrame({
    'student': ['Alice', 'Bob'],
    'math':    [90, 70],
    'english': [85, 80],
})
```

In [26]:
```
wide
```

Out[26]:

	student	math	english
0	Alice	90	85
1	Bob	70	80

In [25]:
```
wide.melt(id_vars=['student'], var_name='subject', value_name='score')
```

Out[25]:

	student	subject	score
0	Alice	math	90
1	Bob	math	70
2	Alice	english	85
3	Bob	english	80

Exercise 7 — putting it together

```
sales = pd.DataFrame({
    'rep':    ['Alice', 'Alice', 'Bob', 'Bob', 'Carol'],
    'region': ['north', 'south', 'north', 'south', 'north'],
    'amount': [200, 150, 300, None, 250],
})
```

- Fill the missing `amount` with the column mean
- Compute total sales per rep

- Return a Series sorted descending by total sales

In [39]:
```
import numpy as np
```

In [41]:
```
np.where(sales["amount"].isna() )[0]
```

Out[41]:
```
array([3])
```

In [45]:
```
sales = pd.DataFrame({
    'rep':    ['Alice', 'Alice', 'Bob', 'Bob', 'Carol'],
    'region': ['north', 'south', 'north', 'south', 'north'],
    'amount': [200, 150, 300, None, 250],
})

# implement this
sales["amount"] = sales["amount"].fillna(sales["amount"].mean())
```

In [52]:
```
total = sales.groupby('rep')["amount"].sum().sort_values(ascending=False)
```

In [53]:
```
total
```

Out[53]:
```
rep
Bob      525.0
Alice    350.0
Carol    250.0
Name: amount, dtype: float64
```

▸ Solution

```
sales['amount'] = sales['amount'].fillna(sales['amount'].mean())
totals = sales.groupby('rep')
['amount'].sum().sort_values(ascending=False)
# rep
# Bob      475.0
# Alice    350.0
# Carol    250.0
```

Part 3 — Python Classes

Basic structure

```
class Dog:
    # __init__ runs when you instantiate: Dog(...)
    def __init__(self, name, breed):
        self.name = name      # instance attribute
        self.breed = breed

    def bark(self):
        return f"{self.name} says woof"

d = Dog("Rex", "labrador")
```

```
print(d.name)    # Rex
print(d.bark())  # Rex says woof
```

- `self` is always the first argument of every method — it refers to the instance
- `__init__` is the constructor; assign instance attributes with `self.x = ...`
- Call methods with `d.bark()`, not `Dog.bark(d)` (though both work)

Class vs instance attributes

```
class Counter:
    count = 0           # class attribute — shared across all
instances

    def __init__(self, name):
        self.name = name    # instance attribute — unique per
instance

    def increment(self):
        Counter.count += 1  # modify class attribute
```

Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "..."

class Cat(Animal):
    def speak(self):            # override parent method
        return f"{self.name} says meow"

    def __init__(self, name, indoor):
        super().__init__(name)     # call parent __init__
        self.indoor = indoor
```

- `super()` calls the parent class — use it in `__init__` to avoid rewriting parent setup
- Overriding a method replaces it; call `super().method()` if you want to extend it

Dunder methods (special methods)

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):           # what print() shows
        return f"Vector({self.x}, {self.y})"
```

```
    def __add__(self, other):     # enables v1 + v2
        return Vector(self.x + other.x, self.y + other.y)

    def __len__(self):            # enables len(v)
        return 2
```

Common dunders: `__repr__` , `__str__` , `__add__` , `__eq__` , `__len__` , `__getitem__`

In [ ]:
```
# Exercise — implement a simple Node and LinkedList class
# Node: stores a value and a pointer to the next node
# LinkedList: stores a head node, supports append(val) and to_list() -> list

class Node:
    def __init__(self, val):
        # implement this
        pass

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, val):
        # add val to the end of the list
        # implement this
        pass

    def to_list(self):
        # return all values as a plain Python list
        # implement this
        pass

ll = LinkedList()
ll.append(1)
ll.append(2)
ll.append(3)
assert ll.to_list() == [1, 2, 3]
print("LinkedList passed")
```

▸ Solution

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, val):
        new_node = Node(val)
        if self.head is None:
            self.head = new_node
            return
```

```
        curr = self.head
        while curr.next is not None:
            curr = curr.next
        curr.next = new_node

    def to_list(self):
        result = []
        curr = self.head
        while curr is not None:
            result.append(curr.val)
            curr = curr.next
        return result
```